SCE 672: Advanced Robotics

Term Project Assesment 1: System Provisioning, Keyboard Teleoperation, Telemetry, and ROS 2 Integration

Instructor: Prof. Aamer Iqbal Bhatti **Marks: 10**
Institution: King Fahd University of Petroleum & Minerals **Due:** September 23rd, 2026 11:59PM

Submission Instructions

You will submit your assignment via the course submission portal. Your submission will consist of:

1. **A single PDF report** containing your written analytical answers, mathematical derivations, terminal logs, and system diagnostic snapshots (denoted by the ✎ symbol). This must be typeset (e.g., LaTeX).
2. **A ZIP archive / GitHub link** containing your complete workspace code (denoted by the 💻 symbol), including your standalone Python SDK teleoperation scripts, ROS 2 launch configurations, and telemetry CSV logs.

Honor Code

- **Collaboration:** You may discuss high-level architecture and Linux configuration with colleagues. However, each PhD student must independently provision their operating system, write their scripts, perform hardware teleoperation, and draft their final report.
- **Repository Privacy:** If you maintain your workspace on GitHub or GitLab, you must keep the repository **private**. Do not publish starter code, driver modifications, or assignment solutions to public branches.
- **Academic Integrity:** Direct unauthorized copying of system configuration scripts, fabrication of hardware ping logs, or misrepresenting simulation data as hardware execution violates university ethical policies.

Introduction

In this assignment, you will deploy and configure the robotics development environment for the laboratory hardware. You will interface directly with the **DJI RoboMaster EP Core** mobile manipulation platform.

The RoboMaster EP features a distributed compute topology:

- The **Motion Controller** (low-level microcontroller) manages closed-loop wheel velocities, chassis odometry integration, chassis LEDs, and onboard bus sensors.
- The **Intelligent Controller** (high-level processing unit running an embedded Linux/Android kernel) interfaces with USB communication and manages the Wi-Fi interface.

Software integration is structured in two progressive paradigms: (1) low-level direct socket communication using the Python SDK client library for real-time keyboard teleoperation and telemetry logging, and (2) middleware orchestration via ROS 2 executing concurrent teleoperation and state estimation. To avoid dependency on local infrastructure routers, this assignment focuses strictly on two physical topologies: **Wired USB (RNDIS)** and **Direct Wireless Hotspot (AP)**.

Problem 1: Host Operating Environment & Middleware (Marks: 2)

Ubuntu 24.04 LTS (Noble Numbat) incorporates Python 3.12 and strictly enforces PEP 668, preventing direct `pip` installs within the global system Python environment. In this problem, you will provision your bare-metal development host and isolate dependencies.

(i) **Operating System Provisioning:**
Install **Ubuntu 24.04 LTS** as a bare-metal dual-boot OS alongside your existing OS (minimum dedicated partition of 80 GB SSD storage). Virtual machines (e.g., VirtualBox, VMware) are **not acceptable** due to USB RNDIS network virtualization latency and hardware packet loss.

(ii) **ROS 2 & Simulation Toolchain:**
Install **ROS 2 Jazzy Jalisco** (`ros-jazzy-desktop-full`) and **Gazebo Harmonic** (`ros-jazzy-ros-gz`). Install required development utilities:

```
$ sudo apt install python3-colcon-common-extensions libopus-dev python3-pip
```

(iii) **Isolated Python Virtual Environment:**
To prevent system Python corruption while allowing the environment to discover ROS 2 binary bindings (`rclpy`), create a virtual environment with system access:

```
$ python3 -m venv --system-site-packages ~/rm_venv
$ source ~/rm_venv/bin/activate
```

Install dependencies and the community SDK fork:

```
$ python -m pip install -i https://test.pypi.org/simple/ rm-libmedia-codec
$ python -m pip install "numpy<2" "numpy-quaternion" pyyaml
$ python -m pip install git+https://github.com/jeguzzi/RoboMaster-SDK.git
```

(iv) **ROS 2 Driver Compilation:**
Clone and build the `robomaster_ros` driver in your local workspace:

```
$ mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
$ git clone https://github.com/jeguzzi/robomaster_ros.git
$ cd ~/ros2_ws && colcon build --symlink-install
```

(v) **ABI Incompatibility Analysis:**
Recent releases of NumPy (2.x) introduce major C-API and Application Binary Interface (ABI) modifications. When running ROS 2 nodes that rely on `cv_bridge` and compiled extension bindings, explain why an unconstrained `pip install -U numpy` triggers catastrophic segmentation faults or `ImportError` crashes. Explain how your virtual environment and shell setup script isolate and resolve this conflict.

Problem 2: Telemetry & Keyboard Telecommand via RoboMaster SDK Marks: 3

Before deploying high-level ROS nodes, you will implement an interactive keyboard teleoperation and telemetry acquisition pipeline using the native Python SDK client library across both physical connection modes.

(i) **Interactive Teleoperation & Telemetry Script:**
Write an independent Python script (`ep_teleop_sdk.py`) that imports the native `robomaster` library (with **zero** ROS 2 dependencies). Your script must implement:

- **Keyboard Telecommand:** Non-blocking keyboard capture (e.g., via `curses` or `pynput`) mapping keys to chassis planar motions: 'W'/'S' for forward/backward (v_x), 'A'/'D' for lateral strafing (v_y), and 'Q'/'E' for yaw rotation (ω_z). Commands must be dispatched to the motion controller via `ep_robot.chassis.drive_speed()`.
- **Telemetry Logging:** Concurrently stream and log timestamped telemetry at 10 Hz to a CSV file: battery state-of-charge, chassis IMU attitude (yaw, pitch, roll), and 4-wheel encoder velocities.
- **Failsafe:** Pressing the `Spacebar` or releasing keys must immediately stop chassis motion ($v_x = v_y = \omega_z = 0$).

(ii) **Wired Mode Teleoperation (RNDIS):**
Set the physical switch on the Intelligent Controller to **Direct/USB**. Connect the robot to your laptop via a verified Micro-USB data cable. Run `ep_teleop_sdk.py` targeting the default RNDIS gateway (`192.168.42.2`). Teleoperate the chassis for 30 seconds while logging data.

(iii) **Wireless Hotspot Teleoperation (AP):**
Toggle the controller switch to **AP mode**. Connect your laptop Wi-Fi directly to the robot's broadcasted SSID (`ROBOMASTER-XXXXXX`). Run `ep_teleop_sdk.py` across the direct wireless link and teleoperate the chassis for 30 seconds.

(iv) **Comparative Teleoperation Latency Analysis:**
Benchmark ping response times ($N = 100$ packets) for both RNDIS and AP modes. Plot the telemetry sample interval ($\Delta t = t_k - t_{k-1}$) and discuss telecommand responsiveness, command lag, and packet jitter observed over wireless AP relative to wired RNDIS.

Problem 3: Wireless Teleoperation and Telemetry in ROS 2 (Marks: 3)

In this problem, you will launch the hardware abstraction layer over the wireless hotspot (AP mode) and execute simultaneous keyboard teleoperation and state odometry logging using ROS 2.

(i) **Driver Initialization over Hotspot:**
With your host PC associated with the robot's Wi-Fi access point, launch the platform driver:

```
$ source ~/rm_setup.sh
$ ros2 launch robomaster_ros ep.launch conn_type:=ap
```

Verify that the node initializes, advertises `/cmd_vel`, and actively publishes telemetry on `/odom` and `/joint_states`.

(ii) **Keyboard Telecommand Pipeline:**
In a secondary terminal, launch a ROS 2 keyboard teleoperation node:

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/cmd_vel
```

Teleoperate the robot along a designated path using both translational longitudinal driving and lateral strafing maneuvers.

(iii) **Telemetry Acquisition:**
Record the `/odom` and `/joint_states` topics while driving using ROS 2 bags:

```
$ ros2 bag record /odom /joint_states /cmd_vel
```

Confirm that data packets are logged continuously without network stalls.

(iv) **State Estimation & Odometry Trajectory Evaluation:**
Extract the recorded `/odom` data and plot the estimated 2D trajectory (X vs. Y). Compare the estimated endpoint against the true physical position of the robot. Discuss the sources of odometry drift, evaluating the influence of Mecanum roller slip and dead-reckoning integration errors.

Problem 4: Theoretical Formulation & System Diagnostics Marks :2

(i) **Network Topology Characterization:**
Complete Table 1 comparing the two deployed communication models based on your empirical observations.

(ii) **Mecanum Kinematic Derivation:**
Consider the 4-wheel Mecanum chassis illustrated in Figure 1. Let R be the wheel radius, l_x the half-wheelbase (longitudinal distance from center to axle), and l_y the half-track width (lateral distance from center to wheel center). The chassis twist is defined as $\mathbf{v}_b = [v_x, v_y, \omega_z]^T$.

Table 1: Physical Connection Topology Comparison

Metric	**Wired USB (`rndis`)**	**Direct Hotspot (`ap`)**
Host Network Interface Name		
Host IP Address & Netmask		
Robot Endpoint IP		
Round-Trip Latency (Avg ± Jitter)		
Host Internet Retention (Yes/No)		
Multi-Robot Scalability		

Derive the forward and inverse kinematic matrices mapping body twist to wheel angular velocities:

$$\begin{bmatrix} \omega_{fl} \\ \omega_{fr} \\ \omega_{rl} \\ \omega_{rr} \end{bmatrix} = \mathbf{J}_{IK} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix}, \qquad \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} = \mathbf{J}_{FK} \begin{bmatrix} \omega_{fl} \\ \omega_{fr} \\ \omega_{rl} \\ \omega_{rr} \end{bmatrix} \tag{1}$$

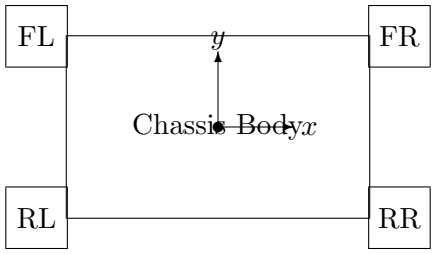


Figure 1: Mecanum chassis geometric layout and wheel indexing conventions.

(iii) **ROS 2 Coordinate Transform Inspection:**
With the driver running, execute:

```
$ ros2 run tf2_tools view_frames
```

Attach the generated TF tree diagram in your report. Identify the parent and child links between `odom`, `base_link`, and `arm_base_link`, and state which node publishes each transform.

Summary of Deliverables

Ensure your final submission contains all items itemized below:

D1. Written Technical Report (PDF format only):

- System diagnostic snapshots verifying Linux kernel, Python, ROS 2 Jazzy, and NumPy versions.
- Detailed explanation of the NumPy 2.x ABI incompatibility and environment isolation methodology (Problem 1).
- Comparative plots of telemetry sampling interval (Δt) and ping round-trip latency for Wired USB vs. Direct AP teleoperation modes (Problem 2).
- Reconstructed 2D odometry trajectory plot (X vs. Y) from the ROS 2 teleoperation run with dead-reckoning slip analysis (Problem 3).
- Fully completed Network Topology Comparison (Table 1).
- Step-by-step mathematical derivation of $\mathbf{J}_{IK}$ and $\mathbf{J}_{FK}$ for the 4-wheel Mecanum drive (Problem 4).
- System TF tree diagram generated via `tf2_tools view_frames` with transform hierarchy explanation.

D2. Codebase & Configuration Repository (ZIP or Private GitHub Link):

- `ep_teleop_sdk.py`: Standalone Python script for keyboard telecommand and telemetry acquisition using the native SDK.
- `rm_setup.sh`: Workspace environment loader script configured with appropriate `PYTHONPATH` and `PYTHONNOUSERSITE`.
- Raw CSV telemetry logs collected during the 30-second RNDIS and AP teleoperation trials.

D3. Video Demonstration Artifact:

- An unlisted YouTube or cloud video link (≤ 45 seconds) clearly demonstrating:
  (a) Keyboard teleoperation of the robot with live terminal telemetry streaming using the native SDK script.
  (b) Driving the robot via ROS 2 keyboard teleoperation over the direct Wi-Fi AP while simultaneously echoing `/odom` in a secondary terminal.
- Demonstrate the above-mentioned Robomaster operation in the class.